

Evaluating Cryptographic API Misuse Detectors for Go

Agreement, coverage, and implications for practice

Vivi Andersson, Martin Monperrus

KTH Royal Institute of Technology, Stockholm, Sweden

April 18, 2026

Rio de Janeiro, Brazil

SVM 2026, co-located with ICSE 2026

Motivation

- We found 7,473 cryptographic API misuse detections across 328 Go projects.
- Many detections are unique to a single tool rather than confirmed by others.
- The main point is that these tools are useful, but different enough that relying on just one can be misleading.

Why This Matters

- Go is heavily used in security-critical infrastructure, cloud systems, and developer tooling.
- Cryptographic API misuse is a high-impact vulnerability class: the code may compile, tests may pass, and the security properties can still be wrong.
- If detectors disagree substantially, both practitioners and evaluators can walk away with the wrong impression of tool quality.

Research Questions and Study Design

- **RQ1.** What is the design space of state-of-the-art tools for detecting cryptographic API misuses in Go?
- **RQ2.** To what extent are crypto misuse detection tools for Go consistent with each other?
- For RQ1, we compare tool scope and rule support. For RQ2, we run all tools on 328 projects and compare their coverage and overlap.

Contribution

First systematic comparison of four Go cryptographic misuse detectors, covering 14 rule classes and 328 security-relevant projects.

RQ1: Taxonomy and Tool Support

Category	IDs	Rules
Cryptographic primitives	01–03	insecure algorithms; insecure PRNG; deprecated Go function
Key management	04–06	constant/predictable key; short key length; static/predictable IV
Password-based KDF	07–09	short salt; predictable salt; low iterations
Transport security	10–11	HTTP protocol; TLS/SSL issues
Secure Shell	12–13	insecure SSH suite; no host key validation
Token auth	14	no JWT verification

- Only rules 02, 05, and 11 are supported by all four tools.
- GOPHER covers the broadest part of the taxonomy, while CODEQL alone supports rule 14.

Dataset and Scope

- 328 security-relevant open-source Go repositories.
- Repository size is strongly right-skewed, with a median of 58,761 LOC.
- Median popularity is 9,096 stars and median contributor count is 167.

Figure unavailable:
output/images/1774885607/repo_metric_loc.png

Figure unavailable:
output/images/1774885607/repo_metric_stars.png

Figure unavailable:
output/images/1774885607/repo_metric_contributors.png

RQ2: Execution Coverage Across Tools

	CodeQL	Gopher	Gosec	Snyk
Projects analyzed (%)	91.8	77.1	100	100
With findings (% of analyzed)	30.9	64.8	84.5	92.7
With findings (% of dataset)	28.4	50.0	84.5	92.7

Interpretation

SNYK CODE and GOSSEC analyze all 328 projects, while GOPHER fails on 75 projects and CODEQL on 27. So even before comparing findings, the tools are operating on different subsets of projects.

RQ2: Cross-Tool Agreement Is Low

- 7,473 detections collapse to 6,152 unique reported locations.
- Only 1.3% of detections are shared by all four tools.
- Most findings are tool-specific: 92.6% of Gosec detections and 63.5% of SNYK CODE detections are unique locations.

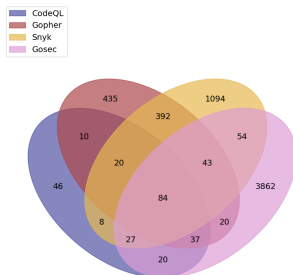
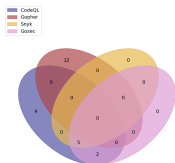


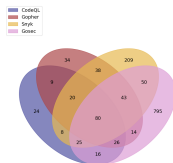
Figure: Overall overlap across tools shows that much of the detection volume is tool-unique.

RQ2: Agreement Depends on the Misuse Rule

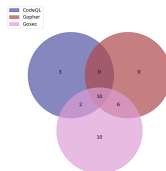
- Rule #05: all four tools support it, but they still disagree sharply.
- Rule #11: the largest class, with 2,033 detections total and 1,049 from GOSec alone.
- Rule #13: only 59 detections total, making manual inspection feasible.



Rule 05



Rule 11



Rule 13

RQ2 takeaway

Agreement varies substantially by misuse class.

Implications for Practice and Evaluation

- **For practitioners:** using a single detector is likely to leave blind spots; using multiple tools is justified by the low overlap.
- **For evaluators:** raw finding counts are misleading when tools disagree this much about the same rule class.
- **For tool builders:** the tools differ in kind, not just in degree: GOPHER covers the broadest rule set, SNYK CODE and GOSEC run most reliably, and CODEQL contributes higher-overlap alerts plus distinct SSH findings.

Limitations and Takeaways

- This study covers four tools, 328 open-source Go projects, and a consolidated mapping into 14 rule classes.
- There are three main caveats: the dataset is time-bounded, the rule mapping requires judgment, and the taxonomy reflects current tool support rather than the full space of possible Go misuses.

Takeaway

The main takeaway is that these tools are not interchangeable. They differ in what they cover, how reliably they run, and how often they agree, so both tool choice and evaluation methodology matter.

Thank you

Questions, discussion, and criticism are welcome.